CS3023SE — Software Engineering

Software Requirements Specification

Intelligent Traffic Management System

Sub-Group B3 — System Engineering & Interaction

Document ID	SRS-B3-001
Version	2.0
Status	Revised — Architecture Aligned
Main Group	Group B — Intelligent Traffic System
Sub-Group	B3 — System Engineering & Interaction
Module	CS3023SE Software Engineering
Date	April 2026

This document follows IEEE Std 830-1998 and incorporates structured Requirements Engineering (RE) practices. Version 2.0 has been revised to align with the confirmed system architecture specification.

Confidential — For Academic Submission Only

Table of Contents

Table of Contents 2
Revision History 4
1. Introduction 5
1.1 Purpose 5
1.2 Scope 5
1.3 Definitions, Acronyms, and Abbreviations 5
1.4 References 6
1.5 Document Overview 7
2. Overall Description 8
2.1 Product Perspective 8
2.2 System Architecture Context 8
2.3 Product Functions — Summary 9
2.4 User Classes and Characteristics 9
2.5 Operating Environment 9
2.6 Assumptions and Dependencies 10
3. Requirements Engineering Process 11
3.1 Elicitation 11
3.1.1 Stakeholder Interviews 11
3.1.2 Scenario Analysis 11
3.1.3 Prototyping 11
3.1.4 Document Analysis 11
3.2 Analysis 11
3.3 Specification 12
3.4 Validation 12
3.5 Requirements Management 12
4. Functional Requirements 13
4.1 Real-Time Traffic Dashboard 13
4.1.1 Dashboard Display 13
4.2 Map-Based Visualisation 13
4.2.1 Interactive Traffic Map 13
4.3 Real-Time Alerting System 14
4.3.1 Alert Generation and Display 14
4.4 Historical Analytics 14
4.4.1 Reporting and Analytics 14

4.5 Administrative Control Panel 14
4.5.1 Admin Functions 14
4.6 Role-Based User Interface 15
4.6.1 UI Routing and Access 15
5. Non-Functional Requirements 16
5.1 Performance 16
5.2 Reliability and Availability 16
5.3 Security 16
5.4 Usability 17
5.5 Maintainability 17
5.6 Scalability 17
6. Domain Requirements 18
7. External Interface Requirements 19
7.1 User Interfaces 19
7.2 Hardware Interfaces 19
7.3 Software Interfaces 19
7.3.1 B2 Data API (Upstream) 19
7.3.2 B4 Authentication & API Gateway (Downstream) 19
7.3.3 PostgreSQL Database 20
7.4 Communication Interfaces 20
8. Use Cases 21
UC-01: View Real-Time Traffic Dashboard 21
UC-02: View and Interact with Traffic Map 21
UC-03: Acknowledge a Critical Alert 21
UC-04: Configure Alert Threshold (Administrator) 22
UC-05: View Historical Traffic Analytics 22
9. Constraints and Limitations 24
9.1 Technical Constraints 24
9.2 Project Constraints 24
9.3 Regulatory and Ethical Constraints 24
10. Requirements Traceability Matrix 25
Appendix A: Glossary of Traffic Domain Terms 26
Appendix B: Requirements Engineering Checklist 27

Revision History

Version	Date	Description	Author
0.1	Apr 01, 2026	Initial draft — scope and stakeholder analysis	B3 Team
0.5	Apr 07, 2026	Functional and non-functional requirements added	B3 Team
1.0	Apr 11, 2026	Final draft submitted for review	B3 Team
2.0	Apr 15, 2026	Revised to align with confirmed system architecture (MQTT, Kafka, Flink, Kong, PostgreSQL, Prometheus/Grafana)	B3 Team

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the complete requirements for the System Engineering and Interaction sub-system (Sub-Group B3) of the Intelligent Traffic Management System (ITMS). It describes the functional requirements, non-functional requirements, constraints, and external interface requirements of the B3 component.

This document is intended for the development team members of B3, other sub-groups (B1, B2, B4) who interface with this sub-system, the module coordinator and academic assessors, and quality assurance personnel conducting review and validation.

1.2 Scope

The B3 sub-system, titled the Traffic Interaction and Visualisation Platform (TIVP), constitutes the complete user-facing layer of the Intelligent Traffic Management System. Its scope covers:

- A real-time traffic monitoring dashboard built on Next.js
- Map-based traffic congestion visualisation using Mapbox GL JS
- Live heatmap rendering for congestion, speed, and incident density
- A real-time push notification and alerting system via Socket.IO
- An administrative control panel for system operators
- A mobile-responsive interface for public traffic awareness

The TIVP does NOT include the following, which are the responsibilities of other sub-groups:

- Raspberry Pi edge devices, camera modules, OpenCV preprocessing, and MQTT client transmission (B1)
- FastAPI + PyTorch/YOLO backend, Apache Kafka stream processing, Apache Flink real-time computation, or data pipelines (B2)
- DigitalOcean Kubernetes (DOKS) cluster orchestration, Kong API gateway, Keycloak authentication, Eclipse Mosquitto MQTT broker, Prometheus/Grafana monitoring, or ArgoCD GitOps deployment (B4)

1.3 Definitions, Acronyms, and Abbreviations

Term	Definition
TIVP	Traffic Interaction and Visualisation Platform (the B3 sub-system)
ITMS	Intelligent Traffic Management System (the full Group B project)
SRS	Software Requirements Specification
FR	Functional Requirement
NFR	Non-Functional Requirement
DR	Domain Requirement
SSR	System-to-System Requirement (inter-subgroup)
UI/UX	User Interface / User Experience
API	Application Programming Interface

REST	Representational State Transfer
SSR (web)	Server-Side Rendering (Next.js feature)
BFF	Backend-for-Frontend (Node.js API layer in B3)
MQTT	Message Queuing Telemetry Transport — lightweight IoT messaging protocol used by B1 edge devices
Kafka	Apache Kafka — distributed event streaming platform used by B2 for high-throughput message brokering
Flink	Apache Flink — stream processing framework used by B2 for real-time congestion calculation
Kong	Kong API Gateway — managed by B4 for routing, rate-limiting, and security enforcement
Mosquitto	Eclipse Mosquitto — MQTT broker deployed by B4, receives edge device transmissions
DOKS	DigitalOcean Kubernetes Service — the managed Kubernetes cluster hosting all services
ArgoCD	GitOps continuous delivery tool watching the traffic-system-gitops repository
WebSocket	Full-duplex communication protocol over TCP
GIS	Geographic Information System
RBAC	Role-Based Access Control
KPI	Key Performance Indicator
ETA	Estimated Time of Arrival
IoT	Internet of Things
RE	Requirements Engineering
YOLO	You Only Look Once — real-time object detection model used by B2 for vehicle detection
PostgreSQL	Relational database storing historical traffic data (deployed in application namespace)

1.4 References

- IEEE Std 830-1998 — Recommended Practice for Software Requirements Specifications
- IEEE Std 29148-2018 — Requirements Engineering
- Sommerville, I. (2016). Software Engineering, 10th Edition. Pearson
- Pressman, R.S. (2014). Software Engineering: A Practitioner’s Approach, 8th Edition
- CS3023SE Module Assignment Brief — Group B Project Specification (2026)
- Next.js Documentation — https://nextjs.org/docs
- Mapbox GL JS Documentation — https://docs.mapbox.com/mapbox-gl-js
- Socket.IO Documentation — https://socket.io/docs
- WCAG 2.1 Accessibility Guidelines — https://www.w3.org/TR/WCAG21
- Apache Kafka Documentation — https://kafka.apache.org/documentation
- Kong Gateway Documentation — https://docs.konghq.com
- Eclipse Mosquitto Documentation — https://mosquitto.org/documentation
- DigitalOcean Kubernetes (DOKS) Documentation — https://docs.digitalocean.com/products/kubernetes

1.5 Document Overview

Section 2 provides the overall system description and context. Section 3 describes the Requirements Engineering process. Section 4 presents Functional Requirements. Section 5 presents Non-Functional Requirements. Section 6 presents Domain Requirements. Section 7 covers external interface requirements. Section 8 covers use cases. Section 9 addresses constraints and assumptions. Section 10 covers requirements validation and traceability.

2. Overall Description

2.1 Product Perspective

The Traffic Interaction and Visualisation Platform (TIVP) is one of four tightly coupled sub-systems within the Intelligent Traffic Management System. It occupies the presentation and interaction tier of the overall architecture, operating as the sole interface between the system and its human users.

The B3 sub-system receives processed data from B2 (Data & Intelligence) via a Kafka consumer and a REST API bridge, where B2 runs FastAPI + PyTorch/YOLO for vehicle detection and Apache Flink for real-time stream processing. B3 communicates with B4 (Platform, Security & Integration) for Keycloak authentication tokens and Kong API gateway routing. It has no direct communication with B1 (Device & Edge Systems); all sensor data from Raspberry Pi edge devices arrives pre-processed through B2 via the Mosquitto MQTT broker and Kafka pipeline.

2.2 System Architecture Context

The overall system follows a five-layer architecture deployed on a DigitalOcean Kubernetes (DOKS) cluster in the BLR1 (Bangalore) region:

Layer 1 — The Edge (B1: Device & Edge Systems)

Raspberry Pi devices mounted at intersections capture live video via camera modules. An edge preprocessor (Python + OpenCV) extracts and compresses frames, optionally runs lightweight YOLO detection, and transmits payloads via an MQTT client to the cloud.

Layer 2 — GitHub (Code Storage + CI/CD Trigger)

A monorepo (traffic-system/) contains all code: edge/ (B1 Pi scripts), backend/ (B2 FastAPI + PyTorch/YOLO), frontend/ (B3 Next.js + Mapbox), and .github/workflows/ (CI/CD pipelines). A separate traffic-system-gitops/ repository is watched by ArgoCD for deployment changes.

Layer 3 & 4 — CI/CD Pipelines (GitHub Actions)

CI pipeline: Pytest → SonarCloud → Docker Build → Trivy Scan → Push to Docker Hub. CD pipeline: Update GitOps repo → ArgoCD detects change → Rolling deployment to Kubernetes.

Layer 5 — DigitalOcean Kubernetes Cluster (B4: Platform & Security)

The DOKS cluster is organised into the following namespaces:

- argocd — The GitOps deployment engine
- api-gateway — Kong API Gateway for routing, security, and rate-limiting
- streaming — Eclipse Mosquitto (MQTT broker receiving Pi data), Apache Kafka (high-throughput message broker), and Apache Flink (real-time stream processing for congestion calculation)
- application — FastAPI + PyTorch pods (B2: reads Kafka, runs YOLO detection, saves to DB), Next.js frontend pods (B3: the UI dashboard), and PostgreSQL database (historical traffic data storage)
- monitoring — Prometheus (metrics scraping) and Grafana (DevOps dashboards)

TIVP Architecture (B3 Component)

Within this architecture, the TIVP follows a modern web architecture pattern:

- Frontend: Next.js application served via SSR, consuming REST and WebSocket APIs, deployed as pods in the application namespace
- BFF Layer: Node.js + Express server that aggregates data from B2's FastAPI endpoints and routes to the frontend
- Real-time channel: Socket.IO server co-located with the BFF layer, consuming events from the Kafka stream
- Map layer: Mapbox GL JS integrated in Next.js for GIS visualisation
- State management: Zustand or Redux Toolkit for client-side state
- Data persistence: PostgreSQL (shared in the application namespace) for historical traffic data

2.3 Product Functions — Summary

The following high-level functions are provided by the TIVP:

1. Real-Time Traffic Dashboard — live congestion and incident visualisation
2. Map-Based Visualisation — interactive Mapbox heatmaps, route overlays, incident markers
3. Multi-Level Alerting System — real-time push notifications for traffic events
4. Historical Analytics — trend charts and comparative time-series analysis using PostgreSQL data
5. Administrative Control Panel — operator tools for managing alerts and system state
6. Public-Facing Passenger View — simplified, mobile-first traffic status interface
7. Role-Based UI Routing — different views for public users, operators, and administrators

2.4 User Classes and Characteristics

User Class	Description	Technical Level	Primary Interface
System Operator	Traffic control centre staff managing live incidents	Intermediate	Operator Dashboard
Administrator	System admins configuring alerts, thresholds, and users	High	Admin Panel
Public User	General public checking traffic conditions	Low	Public Web / Mobile
B4 Platform Team	DevOps team managing deployment, Kong gateway, and Kubernetes	High	API Endpoints, ArgoCD, Grafana

2.5 Operating Environment

- Deployment: Docker containers orchestrated by DigitalOcean Kubernetes (DOKS) in BLR1 region, managed by B4 via ArgoCD GitOps
- Node.js runtime: v20 LTS
- Browsers supported: Chrome 110+, Firefox 110+, Safari 16+, Edge 110+
- Mobile support: iOS 15+, Android 10+ (responsive web)
- Network: WebSocket-capable environment; graceful degradation to HTTP polling

- CI/CD: GitHub Actions (Pytest, SonarCloud, Docker Build, Trivy Scan, Docker Hub push)
- Monitoring: Prometheus metrics scraping with Grafana dashboards (managed by B4)

2.6 Assumptions and Dependencies

- B2 exposes a stable REST API (FastAPI) and Kafka topic for traffic data consumption; B2 runs YOLO detection and Apache Flink stream processing upstream
- B4 provides OAuth2 / Keycloak tokens for authentication; B3 validates these but does not issue them
- Mapbox API key is provisioned and available in the deployment environment
- B4 provides the Kong API gateway (deployed in the api-gateway namespace) through which B3 BFF routes external calls
- B4 deploys and manages the Eclipse Mosquitto MQTT broker, Apache Kafka, and Apache Flink in the streaming namespace
- PostgreSQL database is available in the application namespace for historical data storage
- B1 Raspberry Pi edge devices reliably transmit MQTT payloads; connectivity issues are handled by the Mosquitto broker
- ArgoCD in the argocd namespace watches the traffic-system-gitops repository for deployment updates
- All sub-groups adhere to the agreed inter-group API contract (defined separately)
- SonarCloud quality gate and Trivy security scan pass before deployment proceeds

3. Requirements Engineering Process

This section describes the structured RE process followed by Group B3, aligned with the Sommerville (2016) RE framework, which defines four core activities: Elicitation, Analysis, Specification, and Validation.

3.1 Elicitation

3.1.1 Stakeholder Interviews

Structured interviews were conducted with representatives from each sub-group to understand inter-system dependencies. Key questions focused on data formats exposed by B2's FastAPI endpoints and Kafka topics, authentication flows provided by B4's Keycloak, Kong gateway routing configuration, and alerting triggers from B1 Raspberry Pi devices via B2's MQTT-Kafka-Flink pipeline.

3.1.2 Scenario Analysis

Three primary operational scenarios were analysed to derive requirements: (1) a peak-hour congestion event where Flink processes real-time vehicle density from multiple Pi camera feeds, (2) a road incident requiring multi-level alerting triggered by YOLO detection in B2, and (3) a system operator manually escalating an alert. Each scenario was walked through to identify gaps in the interaction design.

3.1.3 Prototyping

Low-fidelity wireframes of the dashboard, map view, and alert panel were produced using Figma and reviewed by the B3 team. Feedback was used to refine the UI interaction model before specification.

3.1.4 Document Analysis

The project assignment brief, IEEE 830 standard, the confirmed system architecture specification, and Mapbox/Socket.IO/Kafka/Kong documentation were analysed to derive implicit technical requirements and integration constraints.

3.2 Analysis

Elicited requirements were analysed for completeness, consistency, and feasibility. Conflicts identified during analysis:

- Conflict: Public users requested near-instantaneous updates vs. B2's Flink processing latency and Kafka consumer lag. Resolution: TIVP implements a caching layer in the BFF with a 5-second TTL, and Socket.IO pushes delta updates as Kafka events arrive.
- Conflict: Operators required simultaneous multi-map views vs. Mapbox GL performance limits. Resolution: Limit simultaneous map instances to 2; provide toggle view.
- Conflict: Authentication handled by B4 Keycloak vs. B3's need for client-side session state. Resolution: B3 stores JWT in HttpOnly cookies; Kong gateway and B4 validate on each API call.
- Conflict: B3 BFF needs to consume Kafka topics but Kafka is managed in B4's streaming namespace. Resolution: B4 exposes Kafka consumer endpoints via the Kong gateway, and B3 BFF subscribes via the provided connection string.

3.3 Specification

Requirements are specified in this document using the following conventions:

- Functional Requirements: REQ-FR-XXX — defined in Section 4
- Non-Functional Requirements: REQ-NFR-XXX — defined in Section 5
- Domain Requirements: REQ-DR-XXX — defined in Section 6
- Priority levels: P1 (Must Have), P2 (Should Have), P3 (Nice to Have) — aligned with MoSCoW method
- Each requirement is uniquely identified, atomic, verifiable, and traceable to a use case or stakeholder need

3.4 Validation

8. Peer Review: All requirements reviewed by at least one B3 member not involved in drafting
9. Cross-group Review: B2 and B4 leads sign off on inter-system interface requirements, including Kafka topic schemas and Kong gateway routes
10. Prototype Walkthrough: Requirements are traced against wireframes to verify completeness
11. Checklist Inspection: IEEE 830 quality attributes checked for each requirement
12. Requirements Traceability Matrix (RTM): Maintained in Section 10 to ensure all requirements are linked to test cases

3.5 Requirements Management

- All requirements are stored in the shared GitHub monorepo under /docs/requirements/ within the traffic-system/ repository
- Change requests follow the formal CR process defined in the project management plan
- The RTM is updated with every approved change request
- Version control: This SRS follows semantic versioning (MAJOR.MINOR)
- CI/CD pipeline (GitHub Actions) ensures all code changes pass SonarCloud quality gates before merging

4. Functional Requirements

Functional requirements are grouped by feature area. Each requirement is uniquely identified, prioritised (P1/P2/P3), and traceable to a use case in Section 8.

4.1 Real-Time Traffic Dashboard

4.1.1 Dashboard Display

Req ID	Priority	Requirement Statement
REQ-FR-001	P1	The system shall display a real-time traffic dashboard that auto-refreshes at a minimum interval of 5 seconds without requiring a page reload.
REQ-FR-002	P1	The dashboard shall display a summary panel showing: total active incidents, average network speed, congestion level (Low / Moderate / High / Critical), and active alert count.
REQ-FR-003	P1	The dashboard shall display a time-series line chart showing traffic volume for the past 60 minutes, updated in real time via Socket.IO events consumed from Kafka.
REQ-FR-004	P2	The dashboard shall allow users to switch between time range views: Last 1 Hour, Last 6 Hours, Last 24 Hours, and Custom Range.
REQ-FR-005	P2	The system shall display a live feed panel listing the 10 most recent traffic events, each showing location, event type, severity, and timestamp.
REQ-FR-006	P3	The dashboard shall support a dark mode / light mode toggle that persists across sessions via HttpOnly cookie preference (not localStorage, as per security constraints).
REQ-FR-007	P4	

4.2 Map-Based Visualisation

4.2.1 Interactive Traffic Map

Req ID	Priority	Requirement Statement
REQ-FR-007	P1	The system shall render an interactive traffic map using Mapbox GL JS, centred on the city coverage area, with standard pan, zoom, and tilt controls.
REQ-FR-008	P1	The map shall display a live congestion heatmap layer rendered from vehicle density data received from B2 (via FastAPI REST endpoint or Kafka consumer), updated at intervals not exceeding 10 seconds.
REQ-FR-009	P1	The system shall display incident markers on the map for each active traffic event, using colour-coded icons to indicate severity: Green (low), Amber (moderate), Red (high), Purple (critical).
REQ-FR-010	P1	Clicking an incident marker shall open an information panel displaying: incident type, road name, start time, estimated clearance time, affected lanes, and data source (YOLO detection / manual report).
REQ-FR-011	P2	The map shall support layer toggling: users shall be able to independently enable/disable the heatmap layer, incident markers, and route overlay.
REQ-FR-012	P2	The system shall display average speed overlays on major roads using a colour gradient: green (>60 km/h), amber (30-60 km/h), red (<30 km/h).
REQ-FR-013	P2	The map shall support geofencing: administrators shall be able to draw a zone on the map and receive a filtered data feed for that zone only.

REQ-FR-014	P3	The system shall support satellite basemap as an alternative to the default street map.

4.3 Real-Time Alerting System

4.3.1 Alert Generation and Display

Req ID	Priority	Requirement Statement
REQ-FR-015	P1	The system shall receive real-time alert events from B2 via a Socket.IO WebSocket connection (sourced from the Kafka event stream) and display them to the relevant user class within 2 seconds of receipt.
REQ-FR-016	P1	Alerts shall be classified into four severity levels: Informational, Warning, Critical, and Emergency, displayed with distinct visual styles (colour, icon, and sound cue).
REQ-FR-017	P1	The system shall display a persistent alert notification banner at the top of the operator and administrator views for all Critical and Emergency alerts until acknowledged.
REQ-FR-018	P1	Operators shall be able to acknowledge an alert, which removes it from the active alert banner but retains it in the alert history log stored in PostgreSQL.
REQ-FR-019	P2	The system shall support alert filtering by: severity level, road segment, date/time range, and alert type.
REQ-FR-020	P2	The system shall send browser push notifications for Critical and Emergency alerts, provided the user has granted notification permission.
REQ-FR-021	P2	The system shall maintain an alert history log viewable by operators and administrators, persisted in PostgreSQL with a minimum retention of 30 days.
REQ-FR-022	P3	The system shall support alert export to CSV format for operator reporting.

4.4 Historical Analytics

4.4.1 Reporting and Analytics

Req ID	Priority	Requirement Statement
REQ-FR-023	P2	The system shall provide a historical analytics module allowing operators to query traffic data stored in PostgreSQL for any date range within the available data retention period.
REQ-FR-024	P2	The analytics module shall display: average daily congestion index, peak congestion hour distribution (bar chart), top 10 most congested road segments (ranked list), and incident frequency by type (pie chart).
REQ-FR-025	P2	The system shall allow side-by-side comparison of two arbitrary date ranges for all analytics metrics.
REQ-FR-026	P3	The system shall generate a printable PDF report of analytics for a selected date range.

4.5 Administrative Control Panel

4.5.1 Admin Functions

Req ID	Priority	Requirement Statement
REQ-FR-027	P1	The system shall provide an administrative panel, accessible only to users with the Administrator role (validated via Keycloak JWT role claims), for managing alert thresholds and notification rules.

REQ-FR-028	P1	Administrators shall be able to configure threshold values for each alert severity level, specifying the sensor metric (e.g., vehicle count > N triggers Warning), persisted in PostgreSQL.
REQ-FR-029	P2	Administrators shall be able to create, edit, and delete named geographic zones on the map for targeted monitoring.
REQ-FR-030	P2	Administrators shall be able to broadcast a manual system-wide notification message to all active operator sessions via Socket.IO.
REQ-FR-031	P3	The admin panel shall provide a user session management view showing all active operator sessions.

4.6 Role-Based User Interface

4.6.1 UI Routing and Access

Req ID	Priority	Requirement Statement
REQ-FR-032	P1	The system shall enforce role-based UI routing based on Keycloak JWT role claims: Public users see read-only traffic status; Operators see the full dashboard and alert management; Administrators see all operator views plus the admin panel.
REQ-FR-033	P1	The system shall redirect unauthenticated users attempting to access operator or administrator views to the B4-managed Keycloak login page.
REQ-FR-034	P1	The system shall display only the navigation items relevant to the authenticated user's role, hiding inaccessible views entirely.
REQ-FR-035	P2	The public-facing view shall be fully accessible without authentication and shall be optimised for mobile screen sizes (responsive design, min 320px width).

5. Non-Functional Requirements

Non-functional requirements define the quality attributes of the TIVP. Each is uniquely identified and measurable, following ISO/IEC 25010 quality model categories.

5.1 Performance

Req ID	Priority	Requirement Statement
REQ-NFR-001	P1	The dashboard initial page load time shall not exceed 3 seconds on a standard broadband connection (50 Mbps) as measured by Lighthouse performance score.
REQ-NFR-002	P1	Real-time dashboard updates received via Socket.IO shall be rendered on-screen within 500 milliseconds of receipt.
REQ-NFR-003	P1	The Mapbox heatmap layer shall complete a full re-render within 1 second when new data arrives.
REQ-NFR-004	P2	The system shall support a minimum of 500 concurrent WebSocket connections without degradation of update latency beyond 1 second.
REQ-NFR-005	P2	API response time from the BFF layer to frontend requests shall not exceed 200 milliseconds at the 95th percentile under normal load.

5.2 Reliability and Availability

Req ID	Priority	Requirement Statement
REQ-NFR-006	P1	The TIVP shall maintain a minimum availability of 99.5% measured over any 30-day period, excluding planned maintenance windows.
REQ-NFR-007	P1	The Socket.IO client shall implement automatic reconnection with exponential backoff (starting at 1s, max 30s) on connection loss.
REQ-NFR-008	P1	In the event of a BFF, B2 FastAPI, or Kafka consumer failure, the frontend shall display a degraded-mode notice and serve the last successfully cached data to users.
REQ-NFR-009	P2	The system shall implement circuit-breaker logic in the BFF layer to prevent cascading failures when upstream services (FastAPI, Kafka, PostgreSQL) are unavailable.

5.3 Security

Req ID	Priority	Requirement Statement
REQ-NFR-010	P1	All communication between the frontend client and the BFF layer shall use HTTPS/TLS 1.3 or higher. Unencrypted HTTP connections shall be rejected.
REQ-NFR-011	P1	The BFF layer shall validate the JWT bearer token issued by B4 Keycloak on every API call. Expired or invalid tokens shall result in a 401 Unauthorized response. Kong gateway performs additional validation at the API gateway layer.
REQ-NFR-012	P1	The system shall implement Content Security Policy (CSP) headers to prevent cross-site scripting (XSS) attacks.
REQ-NFR-013	P1	All user-supplied inputs on forms (e.g., alert comments, search fields) shall be sanitised server-side before processing or storage in PostgreSQL.
REQ-NFR-014	P2	The BFF API shall implement rate limiting: maximum 100 requests per minute per authenticated user, with a 429 Too Many Requests response on breach. Kong gateway enforces additional rate limiting at the gateway level.

REQ-NFR-015	P2	The system shall not expose internal error stack traces, Kubernetes namespace details, or service identifiers in error responses returned to the client.

5.4 Usability

Req ID	Priority	Requirement Statement
REQ-NFR-016	P1	The user interface shall conform to WCAG 2.1 Level AA accessibility guidelines, including minimum 4.5:1 colour contrast ratio for text elements.
REQ-NFR-017	P1	The dashboard and map views shall be fully functional on screens from 320px (mobile) to 3840px (4K) width without horizontal scrolling.
REQ-NFR-018	P2	A new system operator with basic computer literacy shall be able to navigate to and acknowledge an active alert within 2 minutes without prior training, using only the on-screen UI.
REQ-NFR-019	P2	All interactive elements shall provide visual hover and focus states compliant with WCAG 2.1 Focus Visible criterion.
REQ-NFR-020	P3	The system shall support internationalisation (i18n) framework integration (next-i18next), with English as the default locale.

5.5 Maintainability

Req ID	Priority	Requirement Statement
REQ-NFR-021	P1	The frontend codebase shall achieve a minimum code coverage of 70% through unit and integration tests (Jest + React Testing Library), verified by the CI pipeline.
REQ-NFR-022	P2	All React components shall be documented using Storybook, with at least one story per component covering its primary usage state.
REQ-NFR-023	P2	The BFF API shall expose an OpenAPI 3.0 specification document at /api/docs, auto-generated from code annotations.
REQ-NFR-024	P2	Code shall follow consistent formatting enforced by ESLint and Prettier in the CI/CD pipeline (GitHub Actions); any build with lint errors shall be rejected. SonarCloud quality gate must pass.

5.6 Scalability

Req ID	Priority	Requirement Statement
REQ-NFR-025	P2	The BFF layer shall be stateless, allowing horizontal scaling through Kubernetes pod replication in the application namespace without session affinity.
REQ-NFR-026	P2	Socket.IO shall be configured with a Redis adapter to allow WebSocket connections to be distributed across multiple BFF pod instances within the DOKS cluster.
REQ-NFR-027	P3	The system shall be capable of serving the public-facing view as a statically generated site (Next.js ISR) to support CDN distribution.

6. Domain Requirements

Domain requirements arise from the application domain of intelligent transport systems and impose constraints that stem from professional standards, regulations, and established practices in the traffic management domain.

Req ID	Priority	Requirement Statement
REQ-DR-001	P1	Traffic severity classifications displayed in the system shall conform to the standard four-tier classification: Level 1 (Minor), Level 2 (Moderate), Level 3 (Severe), Level 4 (Critical / Road Closure).
REQ-DR-002	P1	Incident timestamps displayed to operators shall be in UTC with local timezone offset shown in parentheses, to prevent ambiguity in cross-region deployments.
REQ-DR-003	P1	Speed data displayed on maps shall be presented in km/h as the standard unit; conversion to mph shall be available as an optional setting.
REQ-DR-004	P2	The system shall display a data staleness indicator on any metric that has not been updated for more than 30 seconds, clearly communicating to operators that the displayed data may not reflect the current state (e.g., Kafka consumer lag or Flink processing delay).
REQ-DR-005	P2	Geographic coordinates used in incident markers and zone definitions shall be stored and transmitted in WGS84 (EPSG:4326) standard, consistent with standard GIS and GPS systems.
REQ-DR-006	P2	Alert acknowledgement actions shall be logged with the operator's user ID and timestamp and retained in an immutable audit log in PostgreSQL for a minimum of 90 days, to support incident review processes.
REQ-DR-007	P3	The system shall be capable of integrating with standard traffic data feed formats (DATEX II / GTFS-RT) as optional data source connectors, to support future interoperability with external traffic management systems.

7. External Interface Requirements

7.1 User Interfaces

- The frontend shall be a Next.js application with Server-Side Rendering for initial page load, deployed as pods in the DOKS application namespace.
- Navigation shall use a persistent sidebar (desktop) and bottom tab bar (mobile).
- All data visualisation components (charts, maps, tables) shall include accessible text alternatives.
- Error states, loading states, and empty states shall be explicitly designed and implemented for all data-fetching components.

7.2 Hardware Interfaces

- No direct hardware interface is required by B3. All sensor data from Raspberry Pi camera modules arrives pre-processed via B2's FastAPI endpoints, having passed through the Mosquitto MQTT broker, Kafka, and Flink pipeline.
- The system shall detect and adapt to touch-screen input on mobile and tablet devices.
- The system shall support mouse, keyboard, and touch navigation for all interactive elements.

7.3 Software Interfaces

7.3.1 B2 Data API (Upstream)

Property	Description
Interface Type	REST API (FastAPI) + Kafka Consumer
Data Consumed	Traffic event stream, congestion metrics (from Flink), speed data, incident feed (from YOLO detection)
Protocol	HTTPS REST (polling / SSE) via Kong gateway, and Kafka topic subscription via BFF
Auth	JWT Bearer Token (issued by B4 Keycloak, validated by Kong gateway)
Format	JSON over REST; Avro or JSON schema over Kafka
Update Interval	Minimum 5 seconds (configurable per metric)
Data Source Chain	Raspberry Pi (MQTT) → Mosquitto → Kafka → Flink → FastAPI → Kong → B3 BFF

7.3.2 B4 Authentication & API Gateway (Downstream)

Property	Description
Interface Type	OAuth2 / OpenID Connect
Auth Provider	Keycloak (managed by B4)
Token Type	JWT Bearer Token with role claims
Session Handling	HttpOnly cookie; refresh token via /auth/refresh endpoint
API Gateway	Kong (B4, deployed in api-gateway namespace); B3 BFF routes via Kong for all upstream calls

7.3.3 PostgreSQL Database

Property	Description
Interface Type	SQL via Node.js database driver (e.g., pg or Prisma)
Data Stored	Historical traffic data, alert history, audit logs, threshold configurations, geographic zone definitions
Deployment	PostgreSQL pod in the application namespace of DOKS cluster
Access	Internal cluster networking; no external exposure

7.4 Communication Interfaces

- HTTPS (TLS 1.3): All client-server communication, enforced by Kong gateway
- WebSocket (Socket.IO): Real-time event stream from BFF to frontend client, sourced from Kafka events
- REST (JSON): BFF to B2 FastAPI data API (via Kong) and B4 Keycloak authentication API
- Kafka Consumer: BFF subscribes to Kafka topics in the streaming namespace for real-time traffic events
- Redis Pub/Sub: Socket.IO adapter for multi-pod WebSocket distribution (internal to application namespace)
- MQTT (reference only): B1 Raspberry Pi devices transmit to Mosquitto broker; B3 does not directly interface with MQTT

8. Use Cases

The following use cases describe the key interaction scenarios between user classes and the TIVP. Each use case is traceable to one or more functional requirements.

UC-01: View Real-Time Traffic Dashboard

Field	Description
Use Case ID	UC-01
Use Case Name	View Real-Time Traffic Dashboard
Actor(s)	Operator, Administrator, Public User
Precondition	User has loaded the TIVP web application. Operators and Administrators are authenticated via Keycloak.
Trigger	User navigates to the Dashboard view.
Main Flow	1. System displays the traffic summary panel with current congestion level (computed by Flink), speed, and alert count. 2. System renders the time-series traffic volume chart for the past 60 minutes. 3. System establishes a Socket.IO connection and begins receiving real-time updates from Kafka events. 4. Dashboard auto-refreshes all metrics every 5 seconds without page reload. 5. User views the live event feed panel for the most recent 10 incidents.
Alternate Flow	3a. Socket.IO connection fails: System falls back to HTTP polling at 10-second intervals and displays a degraded-mode indicator.
Postcondition	User has an accurate view of current traffic state.
Linked FRs	REQ-FR-001, REQ-FR-002, REQ-FR-003, REQ-FR-005, REQ-NFR-007, REQ-NFR-008

UC-02: View and Interact with Traffic Map

Field	Description
Use Case ID	UC-02
Use Case Name	View and Interact with Traffic Map
Actor(s)	Operator, Administrator, Public User
Precondition	User has an active internet connection. Mapbox API key is valid.
Trigger	User navigates to the Map view.
Main Flow	1. System loads the Mapbox GL map centred on the city area. 2. System fetches current congestion data from BFF (sourced from Flink via FastAPI) and renders the heatmap layer. 3. System places incident markers at reported event locations (detected by YOLO or manually reported). 4. User pans and zooms the map. 5. User clicks an incident marker to view detailed information. 6. User toggles map layers using layer controls.
Alternate Flow	2a. Congestion data not available (Kafka consumer lag or Flink delay): Heatmap layer displays empty with a data staleness indicator.
Postcondition	User has a geospatial understanding of traffic conditions.
Linked FRs	REQ-FR-007, REQ-FR-008, REQ-FR-009, REQ-FR-010, REQ-FR-011, REQ-FR-012, REQ-DR-004, REQ-DR-005

UC-03: Acknowledge a Critical Alert

Field	Description
Use Case ID	UC-03
Use Case Name	Acknowledge a Critical Alert
Actor(s)	Operator, Administrator
Precondition	Operator is authenticated via Keycloak. A Critical or Emergency alert has been received via Socket.IO from the Kafka event stream.
Trigger	A new Critical/Emergency alert arrives and is displayed in the alert banner.
Main Flow	1. System displays an alert banner with alert details and severity indicator. 2. System plays an audible notification cue. 3. Operator reads the alert and clicks Acknowledge. 4. System logs the acknowledgement with operator user ID and timestamp to the PostgreSQL audit log. 5. System removes the alert from the active banner. 6. Alert is retained in the alert history log.
Alternate Flow	3a. Operator clicks the incident location link: System opens the Map view centred on the incident location (UC-02).
Postcondition	Alert is acknowledged; PostgreSQL audit log is updated; alert banner is cleared.
Linked FRs	REQ-FR-015, REQ-FR-016, REQ-FR-017, REQ-FR-018, REQ-DR-006, REQ-NFR-011

UC-04: Configure Alert Threshold (Administrator)

Field	Description
Use Case ID	UC-04
Use Case Name	Configure Alert Threshold
Actor(s)	Administrator
Precondition	User is authenticated with the Administrator role via Keycloak JWT.
Trigger	Administrator navigates to the Admin Panel > Alert Configuration.
Main Flow	1. System displays current threshold configuration (stored in PostgreSQL). 2. Administrator modifies a threshold value. 3. Administrator clicks Save. 4. System validates input (numeric, positive, Warning < Critical < Emergency). 5. System persists to PostgreSQL and confirms with a toast notification.
Alternate Flow	4a. Validation fails: System displays inline validation error without saving.
Postcondition	New threshold configuration is active and applied to subsequent alert evaluations by B2.
Linked FRs	REQ-FR-027, REQ-FR-028, REQ-FR-032, REQ-FR-033, REQ-NFR-013

UC-05: View Historical Traffic Analytics

Field	Description
Use Case ID	UC-05
Use Case Name	View Historical Traffic Analytics
Actor(s)	Operator, Administrator
Precondition	User is authenticated. Historical data is available in PostgreSQL.
Trigger	User navigates to the Analytics module.
Main Flow	1. User selects a date range. 2. System queries BFF which fetches aggregated data from PostgreSQL. 3. System renders: daily congestion index chart, peak hour bar chart, top 10 congested segments table, incident type pie chart. 4. User optionally

	selects a second date range for comparison. 5. System renders both in split-panel view.
Alternate Flow	2a. No data available for selected range: System displays an empty state message.
Postcondition	User has analytical insight into historical traffic patterns.
Linked FRs	REQ-FR-023, REQ-FR-024, REQ-FR-025

9. Constraints and Limitations

9.1 Technical Constraints

- The system must use Next.js as the frontend framework (mandated in the project spec).
- Real-time communication must be implemented using Socket.IO (mandated in the project spec).
- Map visualisation must use Mapbox GL JS (mandated in the project spec).
- The BFF layer must be implemented in Node.js to maintain language consistency and Socket.IO compatibility.
- The system must be containerised using Docker for deployment by B4 via ArgoCD to the DOKS cluster.
- No sensitive data (tokens, credentials) shall be stored in browser localStorage; HttpOnly cookies must be used.
- All API calls from B3 BFF must route through the Kong API gateway in the api-gateway namespace.
- The B3 frontend pods must be deployable in the application namespace alongside FastAPI and PostgreSQL pods.
- Docker images must pass Trivy security scans and SonarCloud quality gates in the CI pipeline before deployment.

9.2 Project Constraints

- Submission deadline: 8 May 2026, 11:55 PM
- Team size: 5 members in B3 sub-group
- No commercial Mapbox tier above the free plan shall be used without explicit approval
- All code must be version-controlled in the shared GitHub monorepo (traffic-system/) under the frontend/ directory
- The system must be demonstrable in a live demo environment on submission day, deployed on the DOKS cluster
- CI/CD pipelines must be defined in .github/workflows/ within the monorepo

9.3 Regulatory and Ethical Constraints

- Traffic data processed by the system shall not include personally identifiable information (PII) about individual vehicle owners or drivers.
- Any ML model output (from B2 YOLO detection or Flink computations) displayed to users shall be clearly labelled as a prediction or estimate, not a confirmed fact.
- The public-facing interface shall comply with WCAG 2.1 Level AA to ensure accessibility for all citizens.

10. Requirements Traceability Matrix

The Requirements Traceability Matrix (RTM) below links each functional requirement to its source use case and planned test type. This is a living document updated with each sprint.

Req ID	Requirement (Summary)	Source UC	Test Type	Status
REQ-FR-001	Dashboard auto-refresh (5s)	UC-01	Integration	Pending
REQ-FR-002	Summary panel metrics	UC-01	Unit	Pending
REQ-FR-003	Time-series chart (60 min, Kafka-sourced)	UC-01	Integration	Pending
REQ-FR-007	Mapbox map render	UC-02	System	Pending
REQ-FR-008	Heatmap layer (Flink data)	UC-02	Integration	Pending
REQ-FR-009	Incident markers (YOLO-detected)	UC-02	Unit	Pending
REQ-FR-010	Incident info panel	UC-02	Unit	Pending
REQ-FR-015	Socket.IO alert delivery (Kafka events)	UC-03	Integration	Pending
REQ-FR-016	Alert severity levels	UC-03	Unit	Pending
REQ-FR-017	Critical alert banner	UC-03	System	Pending
REQ-FR-018	Alert acknowledgement (PostgreSQL log)	UC-03	System	Pending
REQ-FR-027	Admin panel (Keycloak RBAC)	UC-04	System	Pending
REQ-FR-028	Threshold config (PostgreSQL)	UC-04	Integration	Pending
REQ-FR-032	Role-based UI routing (Keycloak JWT)	UC-01-05	System	Pending
REQ-FR-033	Auth redirect (Keycloak login)	UC-01-05	Integration	Pending
REQ-NFR-001	Page load < 3 seconds	All	Performance	Pending
REQ-NFR-002	Socket.IO render < 500ms	UC-01,03	Performance	Pending
REQ-NFR-010	HTTPS / TLS (Kong enforced)	All	Security	Pending
REQ-NFR-011	JWT validation (Keycloak + Kong)	All auth	Security	Pending
REQ-NFR-016	WCAG 2.1 AA compliance	All	Accessibility	Pending

Note: The Status column shall be updated to In Progress, Implemented, or Verified as the project progresses. The RTM is maintained in the GitHub monorepo at /docs/rtm.xlsx.

Appendix A: Glossary of Traffic Domain Terms

Term	Definition
Congestion Index	A normalised score (0–100) representing road network congestion severity, where 0 = free flow and 100 = gridlock. Computed by Apache Flink in real time.
Incident	Any event that causes a reduction in road capacity, including accidents, road works, stalled vehicles, and weather events. Detected by YOLO model or manual report.
Heatmap	A geospatial visualisation technique that uses colour gradients to represent data density — in this context, vehicle density or congestion level.
ETA	Estimated Time of Arrival — predicted time for a vehicle to reach its destination, factoring in current traffic conditions.
Route Overlay	A layer on the traffic map that displays recommended or alternative routes in response to current congestion data.
WebSocket	A persistent, full-duplex TCP connection allowing the server to push data to the browser without the client polling.
BFF	Backend-for-Frontend — a lightweight API server dedicated to serving a specific frontend application, acting as an aggregation and transformation layer.
Edge Device	A Raspberry Pi unit mounted at an intersection with camera module, running OpenCV preprocessing and MQTT transmission (B1 responsibility).
MQTT Broker	Eclipse Mosquitto instance receiving lightweight payloads from edge devices and forwarding to Kafka for stream processing.
Stream Processing	Apache Flink jobs that compute real-time congestion metrics from Kafka event streams.

Appendix B: Requirements Engineering Checklist

Each requirement in this document has been checked against the following IEEE 830 quality attributes:

Quality Attribute	Definition	Verification Method
Correct	Accurately represents the stakeholder's need	Stakeholder sign-off
Unambiguous	Only one possible interpretation	Peer review
Complete	All conditions and responses are specified	Use case walkthrough
Consistent	No internal contradictions	Cross-reference review
Verifiable	Can be tested by a finite, cost-effective process	Test case definition
Modifiable	Structure allows changes without ripple effects	RTM coverage check
Traceable	Origin can be traced to a stakeholder need	RTM (Section 10)

— End of Document —